CAREERPILOT: OBJECT-ORIENTED ANALYSIS AND DESIGN FOR A SMART HIRING AND CAREER DEVELOPMENT PLATFORM

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA1110 – Object Oriented Analysis and Design

to the award of the degree of

BACHELOR OF ENGINEERING IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

SARAN RAJ (192511182)

Under the Supervision of

Dr. A. MANIMARAN

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "CareerPilot: Object-Oriented Analysis and Design for a Smart Hiring and Career Development Platform" has been carried out by SARAN RAJ (192511182) under the supervision of Dr. A. Manimaran and is submitted in partial fulfilment of the requirements for the current semester of the B.E Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr Ramamoorthy
Course Coordinator

Computer Science and Engineering

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. A. Manimaran,
Professor

Computer Science and Engineering

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I G Saranraj of the BE CSE, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "CareerPilot: Object-Oriented Analysis and Design for a Smart Hiring and Career Development Platform" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. External tools and notations used during the project — Unified Modelling Language (UML) diagrams, object-oriented design principles, and any AI-assisted drafting support — have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: [Place]
Date: [Date]

Signature of the Students with Names

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Dhanush R / 192421162	[Use case analysis, actor identification]	[Use case diagram for Smart Hiring module]	[Use case validation]	[Chapter 1]	[20%]
Bheeshma L / 192511332	[Class design]	[Class diagram: Job, FresherProfile, Resume, Skill]	[Class-responsibility review]	[Chapter 2]	[20%]
Sanjay J / 192511331	[Behavioural modelling]	[Sequence diagram: matching workflow]	[Scenario walkthroughs]	[Chapter 3]	[20%]
Saran Raj / 192511182	[Matching-service design]	[MatchingService, MatchResult, Recommendation design]	[Design-pattern verification]	[Chapter 4]	[20%]
Amarnath Varma / 192320013	[Documentation and consolidation]	[Report compilation, diagram consolidation]	[Overall design review]	[Chapter 5, References]	[20%]

Total: 100%

ABSTRACT

CareerPilot is a smart hiring and career development platform conceived to bridge the gap between fresher job seekers and startup recruiters through structured, object-oriented software design. The platform is organised into cooperating modules, of which the Smart Hiring and Recruitment Management module forms the intelligent core: it compares fresher profiles and resumes against startup job requirements to identify suitable opportunities and candidates. This report applies Object-Oriented Analysis and Design (OOAD) methodology to model this module, defining the key classes — Job, FresherProfile, Resume, Skill, MatchingService, MatchResult and Recommendation — along with their responsibilities, relationships and collaborations. The module supports startup job creation and management, public and private job posting, job search and filtering, resume analysis, job-description analysis, skill extraction, fresher-to-job matching, match-percentage calculation, matched- and missing-skill identification, and both job recommendations for freshers and candidate recommendations for startups. The design deliberately applies core OOAD concepts: abstraction, by exposing the matching algorithm through a dedicated MatchingService rather than embedding it in controllers; encapsulation, by keeping scoring rules and similarity calculations inside the matching layer; association, by modelling the evaluation of fresher profiles against jobs as an explicit relationship; and a strategy-oriented design that allows the matching implementation to evolve from weighted rules to semantic or NLP-based matching without redesigning the rest of the system. The resulting design provides an extensible, maintainable foundation for an intelligent recruitment platform while remaining fully documented through UML use-case, class and sequence diagrams.

Keywords: Object-Oriented Analysis and Design, UML, Smart Hiring, Recruitment Management, Resume Analysis, Skill Matching, Strategy Pattern, Career Development Platform.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
OOAD	Object-Oriented Analysis and Design	—
UML	Unified Modelling Language	—
NLP	Natural Language Processing	—
M%	Match percentage between a fresher profile and a job	%
JD	Job Description	—
ATS	Applicant Tracking System	—
CRUD	Create, Read, Update, Delete	—

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Fresh graduates entering the job market and early-stage startups looking to hire both face a matching problem: freshers struggle to discover startup openings suited to their skills, while startups receive large volumes of unfiltered applications and lack the resources of large enterprises to screen resumes manually. Manual resume screening is slow, inconsistent, and prone to overlooking well-suited candidates whose resumes are not worded in the same terms as the job description. CareerPilot is conceived as a smart hiring and career development platform to address this two-sided matching problem, and this report focuses on modelling its intelligent core — the Smart Hiring and Recruitment Management module — using Object-Oriented Analysis and Design (OOAD).

1.2 Need for the Project

- Why the problem needs to be addressed: Keyword-based or manual matching between resumes and job descriptions is time-consuming for startups and unreliable for freshers, causing suitable candidates and suitable jobs to go undiscovered.
- Who is affected: Fresher job seekers who cannot easily find startup roles matching their skill set; startup recruiters who lack dedicated applicant-tracking resources; the platform operator, whose value proposition depends on match quality.
- Existing limitations: General-purpose job portals apply generic, one-size-fits-all filtering and do not expose an extensible matching engine that a small platform team can evolve over time.
- Engineering significance: A well-structured object-oriented design, with matching logic isolated behind a dedicated service and modelled through clear class responsibilities, allows the platform to start with simple rule-based matching and evolve toward more sophisticated techniques without a costly redesign.

1.3 Problem Statement

Given that fresher profiles and resumes must be compared against a growing and varied set of startup job requirements, design an object-oriented software structure for the Smart Hiring and Recruitment Management module that: (i) represents job postings, fresher profiles, resumes and skills as well-defined, cohesive classes, (ii) isolates the matching algorithm so that its internal logic can change (from weighted rule-based scoring to future semantic/NLP-based scoring) without requiring changes to controllers or client code, (iii) supports both directions of recommendation — jobs for freshers and candidates for startups — from the same underlying match computation, and (iv) remains maintainable and extensible as new job-posting types, skill taxonomies, or matching strategies are introduced.

1.4 Project Aim

To apply Object-Oriented Analysis and Design methodology to design the Smart Hiring and Recruitment Management module of CareerPilot, producing a coherent set of classes, use-case models and interaction diagrams that support resume analysis, job-description analysis, skill-based matching, and recommendation generation for both freshers and startups.

1.5 Project Objectives

1. To identify the actors and use cases involved in startup job creation and management, public/private job posting, job search and filtering, and resume submission.

2. To design the core domain classes — Job, FresherProfile, Resume and Skill — with clearly defined attributes and responsibilities.

3. To design a MatchingService class that encapsulates resume analysis, job-description analysis, skill extraction, and match-percentage calculation behind a single, abstracted interface.

4. To model the outputs of matching — MatchResult (including matched and missing skills) and Recommendation (job recommendations for freshers and candidate recommendations for startups) — as distinct, purpose-built classes.

5. To apply and document core OOAD concepts — abstraction, encapsulation, association and a strategy-oriented design — within the module's class and interaction diagrams.

6. To validate the design against representative use-case scenarios (e.g., a fresher applying to a job, a startup reviewing ranked candidates) using sequence diagrams.

1.6 Scope

- Included: OOAD artefacts (use-case diagram, class diagram, sequence diagram) and class-level design for the Smart Hiring and Recruitment Management module — Job, FresherProfile, Resume, Skill, MatchingService, MatchResult and Recommendation.
- Excluded: Front-end UI implementation, persistence-layer (database) implementation details, authentication/authorization subsystems, and the design of other CareerPilot modules outside Smart Hiring and Recruitment Management.
- Assumptions: Fresher profiles and resumes, and startup job postings, are already captured in a structured form accessible to the module; skill extraction can initially rely on keyword/rule-based techniques with later substitution by NLP-based techniques.
- Boundary conditions: The module computes match results and recommendations per fresher–job pair or per job's applicant pool; it does not itself perform the final hiring decision, which remains with the startup recruiter.

1.7 Expected Engineering Outcome

The expected outcome is a validated object-oriented design — expressed through use-case, class and sequence diagrams and supported by class-responsibility documentation — for the Smart Hiring and Recruitment Management module, ready to guide implementation of resume analysis, job matching, and recommendation features within the broader CareerPilot platform.

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Existing solution categories were identified by reviewing publicly known approaches used by general-purpose job portals and applicant tracking systems, published literature on object-oriented software design methodology (use-case driven analysis, UML modelling, and the Strategy design pattern), and general concepts in resume-parsing and skill-matching systems.

2.2 Review of Existing Work

Large general-purpose job portals typically offer keyword search and filter-based discovery, placing the burden of matching on the job seeker or recruiter rather than the system; their matching logic, where present, is usually a proprietary, tightly coupled component embedded deep within a large monolithic application, making it difficult to study, extend, or replace. Applicant Tracking Systems (ATS) used by larger companies apply resume-keyword screening against a job description, but are generally designed for high-volume enterprise hiring and are not tailored to the fresher–startup context, where job descriptions and resumes are often less standardised. From a software-design perspective, the classical Object-Oriented Analysis and Design literature (use-case driven design, the Strategy design pattern, and the general principle of separating an algorithm's interface from its implementation) provides a well-established approach for building a matching component whose internal scoring logic can evolve independently of the rest of the system — a pattern well suited to a platform that expects to move from simple rule-based matching to more advanced techniques over time.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
General job-portal keyword search	Simple, familiar to users	Places matching burden on the user; no structured scoring	Retained only as inspiration for search/filter use cases, not for matching design
Enterprise ATS (resume-keyword screening)	Automates initial screening at scale	Tuned for large enterprises; rigid keyword rules; not fresher/startup-specific	Skill-extraction concept reused, but tailored to fresher/startup vocabulary
Monolithic matching logic embedded in controllers	Fast to build initially	Hard to test, extend, or replace matching logic independently	Explicitly avoided; motivates a dedicated MatchingService class
Strategy-pattern-based matching design (OOAD literature)	Matching algorithm can be swapped without changing client code	Requires more careful upfront design	Directly adopted for CareerPilot's MatchingService

2.4 Research / Engineering Gap

Existing job portals and ATS solutions are optimised either for scale (enterprise ATS) or for generality (broad job portals), and neither is designed around a cleanly separated, swappable matching component suited to a small platform serving freshers and startups. The gap is an absence of an explicitly object-oriented, strategy-based matching design tailored to this fresher–startup context, where the matching approach is expected to evolve from simple rule-based scoring to semantic/NLP-based scoring as the platform matures.

2.5 Proposed Contribution

The proposed design contributes a clean separation between domain data classes (Job, FresherProfile, Resume, Skill), the matching behaviour (encapsulated in MatchingService), and the results consumed by the rest of the platform (MatchResult, Recommendation). By exposing matching through a single service interface rather than embedding scoring logic in controllers, the design directly enables the module's own stated evolution path — from weighted rule-based matching to semantic/NLP-based matching — without requiring redesign of the surrounding system.

CHAPTER 3
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Fresher (job seeker)	Discover startup jobs matching their skills; understand missing skills for a target job
Startup recruiter	Post jobs (public/private); receive ranked, relevant candidate recommendations
Platform administrator	Maintainable, extensible matching logic that can be improved over time

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Create and manage startup job postings	Functional	Job can be created, updated and marked public/private
Search and filter jobs	Functional	Freshers can search/filter jobs by role, skill or location
Analyse resumes and job descriptions to extract skills	Functional	Skill list extracted for every submitted resume and job
Compute a match percentage between a fresher and a job	Functional	MatchResult returned for every evaluated fresher–job pair
Identify matched and missing skills	Functional	MatchResult lists both matched and missing skills explicitly
Generate recommendations for freshers and startups	Functional	Recommendation list returned for a fresher or for a job's applicant pool
Extensibility of matching logic	Non-Functional	Matching strategy can be replaced without modifying calling code
Explainability	Non-Functional	Every match result traceable to specific matched/missing skills

3.2 Constraints & Applicable Standards / Conventions

Standard / Convention	Requirement	How Applied in This Project
UML 2.x notation	Consistent, standard diagram notation for use-case, class and sequence diagrams	All diagrams in this report follow standard UML notation
Object-oriented design principles (SOLID, GRASP)	Single-responsibility, low coupling, high cohesion	MatchingService isolates matching responsibility from Job/FresherProfile/Resume data classes
Data privacy expectations for personal data (resumes, contact details)	Minimise exposure of personal data beyond its intended use	Resume and FresherProfile classes model only fields required for matching and display

3.3 Design Specifications — Class Responsibility Summary

Class	Key Responsibility	Representative Attributes	Representative Methods
Job	Represents a startup job posting	title, description, requiredSkills, visibility (public/private)	createJob(), updateJob(), setVisibility()
FresherProfile	Represents a fresher job seeker's profile	name, skills, education, preferences	updateProfile(), attachResume()
Resume	Represents an uploaded resume document	rawText, extractedSkills, fresherId	parseResume(), extractSkills()
Skill	Represents an individual skill entity	name, category	matchesSkill(otherSkill)
MatchingService	Encapsulates matching algorithm; abstraction point for all matching logic	(stateless / strategy-holding)	computeMatch(profile, job), rankCandidates(job), rankJobs(profile)
MatchResult	Holds the outcome of a single fresher–job match computation	matchPercentage, matchedSkills, missingSkills	getSummary()
Recommendation	Holds a ranked recommendation list for a fresher or a job	targetType, rankedList	getTopN(n)

3.4 Use Case & Behavioural Design

Primary actors: Fresher, Startup Recruiter, Platform Administrator.

- Fresher: Search jobs, view job details, submit/update resume, view match percentage and missing skills, receive job recommendations.
- Startup Recruiter: Create/manage job postings, set visibility (public/private), view ranked candidate recommendations for a job.
- Platform Administrator: Monitor and maintain the matching configuration (e.g., which matching strategy is active).

[Insert Figure 3.1 — Use Case Diagram for the Smart Hiring and Recruitment Management module here.]

[Insert Figure 3.2 — Class Diagram showing Job, FresherProfile, Resume, Skill, MatchingService, MatchResult and Recommendation, with association, aggregation and dependency relationships, here.]

Sequence overview — Fresher-to-job matching workflow: (1) Fresher submits or updates a Resume; (2) Resume.extractSkills() populates the extracted skill list; (3) Fresher requests a match against a selected Job, invoking MatchingService.computeMatch(profile, job); (4) MatchingService compares FresherProfile/Resume skills against Job.requiredSkills and produces a MatchResult containing the match percentage, matched skills and missing skills; (5) MatchingService.rankJobs(profile) aggregates MatchResult objects across available jobs into a Recommendation for the fresher; the symmetric flow, rankCandidates(job), produces a Recommendation of ranked candidates for the startup recruiter.

[Insert Figure 3.3 — Sequence Diagram for the fresher-to-job matching workflow described above.]

3.5 OOAD Concepts Applied

- Abstraction: The matching algorithm is exposed through a MatchingService interface rather than being embedded directly inside controllers, so calling code depends only on what matching does (compute a match, rank candidates, rank jobs), not on how it is computed internally.
- Encapsulation: Scoring rules and similarity calculations are kept entirely inside the matching layer (MatchingService and its internal strategy); Job, FresherProfile and Resume do not need to know how matching scores are derived.
- Association: FresherProfile objects are evaluated against Job objects through an explicit association realised via MatchingService, rather than either class directly referencing the other's internal matching logic.
- Strategy-oriented design: The concrete matching implementation (currently intended to be weighted-rule-based) is designed to be replaceable — for example, with a semantic or NLP-based matching strategy — without redesigning MatchingService's public interface or any of the classes that depend on it.

3.6 Alternative Design Approaches & Evaluation

Alternative 1: Embed matching/scoring logic directly inside Job or a job-search controller.

Alternative 2: Implement matching as a set of free-standing utility functions with no owning class.

Alternative 3 (Selected): A dedicated MatchingService class, following a strategy-oriented design, that owns matching behaviour and returns explicit MatchResult/Recommendation objects.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Cohesion / Single Responsibility	30%	Low	Medium	High
Extensibility (swap matching strategy later)	30%	Low	Medium	High
Testability	20%	Low	Medium	High
Simplicity for a first version	20%	High	Medium	Medium

3.7 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Location of matching logic	Embed inside Job/controller (Alternative 1)	Fewer classes initially	Poor separation of concerns; hard to evolve matching approach	Dedicated MatchingService (Alternative 3) chosen to isolate matching behaviour
Skill extraction technique	Full NLP-based extraction from day one	Higher accuracy on varied resume wording	Significantly higher implementation complexity for an initial version	Rule/keyword-based extraction chosen initially, with the design explicitly allowing a later swap to NLP-based extraction

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
MatchResult granularity	Return only an aggregate match percentage	Simpler result object	No visibility into which specific skills are missing	MatchResult chosen to include matched and missing skill lists for explainability

3.8 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	A cleanly separated matching component reduces future rework and engineering effort as the matching approach evolves	Strategy-oriented design isolates change to a single class	Favoured the dedicated MatchingService over embedded matching logic
Ethics	Automated matching and recommendation could unintentionally disadvantage certain fresher profiles if scoring rules are poorly designed	Matching currently based on explicit, inspectable skill comparison rather than opaque scoring	Recommend periodic review of matching rules for fairness before production use
Health & Safety	Not directly applicable to this software design	N/A	No physical safety mitigation required
Security	Resumes and fresher profiles contain personal data that must be protected	Resume and FresherProfile classes model only fields needed for matching/display, limiting unnecessary data exposure	Recommend access control and encryption at the persistence layer when implemented

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Keyword-based skill extraction misses synonyms or differently worded skills	High	Medium	High	Plan explicit future migration to semantic/NLP-based matching strategy	Medium
Poorly designed match scoring unintentionally disadvantages some fresher profiles	Medium	High	High	Keep matched/missing skills explicit and auditable; periodic fairness review	Medium
Tight coupling between matching logic and controllers (if Alternative 1 had been chosen)	Low (mitigated by design)	Medium	Low	MatchingService abstraction adopted specifically to avoid this coupling	Low
Resume/personal data exposure	Low	High	Medium	Limit stored fields to those needed for matching/display; secure persistence layer at implementation time	Low

CHAPTER 4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The module was developed using a use-case-driven OOAD approach: actors and use cases were identified first, followed by identification of domain classes (Job, FresherProfile, Resume, Skill), then design of the behavioural MatchingService and its outputs (MatchResult, Recommendation), and finally validation of the design through sequence diagrams for the primary matching workflow.

4.2 Design Realisation & Class Outlines

The classes below outline the intended structure of the Smart Hiring and Recruitment Management module. They are presented as design-level class outlines (attributes and method signatures) rather than a full implementation, consistent with the OOAD scope of this report.

Class	Purpose	Design Notes
Job	Domain entity for a startup job posting	Holds requiredSkills as a collection of Skill; visibility flag for public/private posting
FresherProfile	Domain entity for a fresher's profile	Aggregates a Resume; holds declared skills and preferences
Resume	Domain entity for an uploaded resume	extractSkills() populates a Skill collection from raw resume text
Skill	Value-like entity representing a single skill	Supports equality/similarity comparison used during matching
MatchingService	Behavioural class encapsulating the matching algorithm	computeMatch(), rankCandidates(), rankJobs() form its public interface; internal scoring strategy is swappable
MatchResult	Data-holding class for a single match outcome	Stores matchPercentage plus matchedSkills and missingSkills collections
Recommendation	Data-holding class for a ranked list of results	Used both for job recommendations to freshers and candidate recommendations to startups

[Insert prototype screenshot / UML tool export of the finalised class diagram here.]

4.3 Test Plan & Verification (Design-Level)

Requirement	Test Method	Acceptance Criterion	Status (Pass/Fail)
Fresher with fully matching skills against a job	Design walkthrough / scenario trace	MatchResult.matchPercentage = 100%; missingSkills empty	[Pass]
Fresher with partially matching skills	Design walkthrough / scenario trace	MatchResult lists correct matchedSkills and missingSkills	[Pass]
Startup requests ranked candidates for a job	Design walkthrough / scenario trace	Recommendation returns candidates ordered by matchPercentage	[Pass]

Requirement	Test Method	Acceptance Criterion	Status (Pass/Fail)
Job marked private	Design walkthrough / scenario trace	Job excluded from public search/filter use case	[Pass]
Matching strategy replaced (e.g., rule-based → semantic)	Design walkthrough / scenario trace	No change required to MatchingService's public interface or calling classes	[Pass]

4.4 Results

[Insert results table / UML tool screenshots from the finalised diagrams here.] A representative worked example of the matching computation is summarised below.

Scenario	Job Required Skills	Fresher Skills	Matched Skills	Missing Skills	Match %
Strong fit	Python, SQL, Git	Python, SQL, Git, React	Python, SQL, Git	None	100%
Partial fit	Python, SQL, Docker	Python, SQL	Python, SQL	Docker	67%

4.5 Design Analysis & Engineering Judgment

The class-responsibility split between data classes (Job, FresherProfile, Resume, Skill) and the behavioural MatchingService satisfies the single-responsibility principle: data classes do not need to change if the matching algorithm changes, and MatchingService does not need to change if new attributes are added to a Job or FresherProfile that are unrelated to matching. One design risk worth flagging explicitly is that, in this initial design, skill extraction and matching are expected to rely on keyword/rule-based comparison; this will under-match resumes that describe equivalent skills using different terminology (e.g., "JS" vs "JavaScript"). The strategy-oriented design of MatchingService is specifically intended to make this a contained, future upgrade rather than a structural weakness, but it remains a real limitation of the current design until a semantic/NLP-based strategy is implemented.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Actor and use-case identification	Use-case diagram covering Fresher, Startup Recruiter and Administrator	Fully Achieved
Core domain class design (Job, FresherProfile, Resume, Skill)	Class diagram with attributes and responsibilities	Fully Achieved
MatchingService abstraction	MatchingService interface with computeMatch/rankCandidates/rankJobs	Fully Achieved
MatchResult and Recommendation design	Class outlines with matched/missing skills and ranked lists	Fully Achieved
Application of OOAD concepts	Abstraction, encapsulation, association and strategy-oriented design documented in Section 3.5	Fully Achieved
Validation via sequence diagram	Fresher-to-job matching workflow traced step by step	Fully Achieved at design level; not yet validated

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
		against a working implementation

4.7 Strengths & Limitations

Strengths:

- Clear separation between domain data (Job, FresherProfile, Resume, Skill) and matching behaviour (MatchingService), following sound OOAD principles.
- Explicit MatchResult with matched/missing skills gives an explainable output rather than an opaque score.
- Strategy-oriented design explicitly supports the module's own stated evolution path from rule-based to semantic/NLP-based matching.

Limitations:

- The design as described relies on keyword/rule-based skill matching, which will under-match resumes using different terminology for equivalent skills.
- The design is presented at the class/interaction level; it has not yet been validated against a working code implementation or real resume/job data.
- Fairness of the matching/recommendation logic has not been formally evaluated and should be reviewed before production use.
- Persistence, authentication and other cross-cutting concerns are out of scope and will need their own design work during implementation.

4.8 Project Planning, Roles & Budget

[Insert Gantt chart / milestone timeline here.]

Student	Assigned Responsibility	Actual Contribution
Dhanush R	[Use-case analysis and actor identification]	[20%]
Bheeshma L	[Class diagram design]	[20%]
Sanjay J	[Sequence/behavioural modelling]	[20%]
Saran Raj	[MatchingService / MatchResult / Recommendation design]	[20%]
Amarnath Varma	[Documentation and consolidation]	[20%]

Item	Estimated Cost	Actual Cost
UML modelling tool / software license (if any)	[₹ / hrs]	[₹ / hrs]
Documentation and design-review effort	[hrs]	[hrs]

CHAPTER 5
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project applied Object-Oriented Analysis and Design methodology to the Smart Hiring and Recruitment Management module of the CareerPilot platform, addressing the engineering problem that fresher-to-startup job matching requires a maintainable, extensible design rather than ad-hoc keyword filtering or matching logic embedded directly in controllers. The resulting design defines seven core classes — Job, FresherProfile, Resume, Skill, MatchingService, MatchResult and Recommendation — with matching behaviour deliberately isolated behind a MatchingService abstraction. Core OOAD concepts (abstraction, encapsulation, association, and a strategy-oriented design) were explicitly applied and documented, and the design was validated at the use-case and sequence-diagram level against representative fresher-to-job matching scenarios. The work demonstrates a sound, extensible object-oriented foundation for the module, while also identifying concrete limitations — chiefly the reliance on keyword-based skill matching in the initial design — that should be addressed as the platform evolves.

5.2 Future Work

- Implement and evaluate a semantic or NLP-based matching strategy within MatchingService, replacing or supplementing the initial keyword/rule-based approach.
- Design and implement the persistence layer for Job, FresherProfile and Resume, including appropriate access control for personal data.
- Conduct a fairness review of the matching and recommendation logic before production deployment.
- Extend the class design to support richer job-description parsing (e.g., experience-level and location constraints) beyond skill lists.
- Integrate the Smart Hiring and Recruitment Management module with the other planned CareerPilot modules within a consistent overall architecture.
- Validate the design against a working prototype implementation and real fresher/job data.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical application of use-case-driven OOAD to a real-world, two-sided matching problem.
- Design of a strategy-oriented service class that isolates an algorithm expected to evolve over time.
- Modelling of behavioural workflows using UML sequence diagrams to validate a class design before implementation.

Engineering & Professional Skills Developed:

- Balancing initial simplicity against long-term extensibility when designing a matching component.
- Critical evaluation of a design's own limitations (keyword-based matching, unvalidated fairness) before implementation begins.
- Collaborative division of OOAD deliverables (use-case, class and sequence diagrams) across a five-member team.

Individual Reflection (each student, 4–5 lines):

- Most difficult design decision encountered and how it was resolved: [to be completed by student].
- A key OOAD concept applied personally and the reasoning behind it: [to be completed by student].
- New knowledge acquired independently: [to be completed by student].
- What would be redesigned if repeated: [to be completed by student].

REFERENCES

[1] G. Booch, R. A. Maksimchuk, M. W. Engle, B. J. Young, J. Conallen, and K. A. Houston, Object-Oriented Analysis and Design with Applications, 3rd ed., Addison-Wesley, 2007.

[2] C. Larman, Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 3rd ed., Prentice Hall, 2004.

[3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.

[4] Object Management Group (OMG), Unified Modeling Language (UML) Specification, Version 2.5.1, 2017.

[5] I. Jacobson, G. Booch, and J. Rumbaugh, The Unified Software Development Process, Addison-Wesley, 1999.

[Additional literature, standards, and AI-assisted resources used should be added here in consistent IEEE style.]

APPENDICES

Appendix A – Detailed Calculations: [Insert worked numerical example of match-percentage computation for a sample fresher–job pair.]

Appendix B – Engineering Drawings / Schematics: [Insert finalised use case, class and sequence diagrams referenced in Section 3.4.]

Appendix C – Module Reference: Module 3 — Smart Hiring and Recruitment Management

Purpose: This is the intelligent core of CareerPilot. It compares fresher profiles and resumes with startup job requirements to identify suitable opportunities and candidates.

Main Features:

- Startup job creation and management
- Public/private job posting
- Job search and filtering
- Resume analysis
- Job-description analysis
- Skill extraction
- Fresher-to-job matching
- Match percentage calculation
- Matched skill identification
- Missing skill identification
- Job recommendations for freshers
- Candidate recommendations for startups

OOAD Design / Main Classes: Job, FresherProfile, Resume, Skill, MatchingService, MatchResult, Recommendation.

OOAD Concepts Used:

- Abstraction: the matching algorithm is exposed through a MatchingService rather than embedded in controllers.
- Encapsulation: scoring rules and similarity calculations remain inside the matching layer.
- Association: Fresher profiles are evaluated against Jobs.
- Strategy-oriented design: the matching implementation can later change from weighted rules to semantic/NLP matching without redesigning the whole system.

Appendix D – Datasheets: Not applicable to this design-focused project.

Appendix E – Experimental / Raw Data: [Insert sample resume/job-description pairs used to validate the matching design.]

Appendix F – Ethics / Institutional Approval: [Where applicable.]

Appendix G – Project Meeting / Progress Records: [Insert meeting log.]

Appendix H – User/Industry Feedback: [Where applicable.]

Appendix I – Publications / Patents / Competitions / Product Outcomes: [Where applicable.]

Appendix J – Individual Contribution Evidence: [Mandatory for team projects — attach commit history / logbook extracts.]

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.8)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.8)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.8)
Security considerations	Relevant SO	Ch. 3 (3.8)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.4)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)